UNIVERSIDADE DO OESTE PAULISTA

DESIGN DE SOFTWARE

GABRIEL RIBEIRO FERREIRA

RELATÓRIO TÉCNICO: DIAGNÓSTICO DE DESIGN E REFATORAÇÃO DO SISTEMA TECHSTORE

PRESIDENTE PRUDENTE

2026

SUMÁRIO

1 INTRODUÇÃO..... 3
2 CRITÉRIOS E FUNDAMENTOS DE ANÁLISE..... 3
2.1 Coesão..... 3
2.2 Acoplamento..... 4
2.3 Responsabilidades mal distribuídas..... 4
2.4 Dificuldades de manutenção e testes..... 4
3 AMBIENTE E MÉTODO DE ANÁLISE..... 5
4 VISÃO GERAL DO DESIGN ORIGINAL..... 5
5 DIAGNÓSTICO E SOLUÇÃO POR ARTEFATO..... 7
5.1 Artefato da camada de serviço (OrderService)..... 7
5.1.1 Objetivo atual..... 7
5.1.2 Baixa coesão..... 7
5.1.3 Alto acoplamento..... 7
5.1.4 Solução..... 7
5.2 Artefatos da camada de domínio (Order, Customer, Address, OrderItem, Product)..... 8
5.2.1 Objetivo atual..... 8
5.2.2 Baixa coesão..... 8
5.2.3 Alto acoplamento..... 8
5.2.4 Solução..... 8
5.3 Artefatos da camada de infraestrutura (ProductRepository, OrderRepository, index.js)..... 8
5.3.1 Objetivo atual..... 8
5.3.2 Baixa coesão..... 9
5.3.3 Alto acoplamento..... 9
5.3.4 Solução..... 9
6 TESTES AUTOMATIZADOS E EVIDÊNCIAS DE EXECUÇÃO..... 9
7 CONCLUSÃO..... 10
8 REFERÊNCIAS..... 12

1 INTRODUÇÃO

Este relatório foi elaborado no contexto da primeira atividade da disciplina de Design de Software. O objeto de estudo é o TechStore, uma aplicação em Node.js desenvolvida em JavaScript e majoritariamente orientada a objetos. O sistema tem como funcionalidade principal a criação de pedidos e suas regras de negócio como o processamento de pagamentos, a atualização de estoque e o envio de notificações aos clientes.

Conforme estipulado pelo enunciado da atividade, o trabalho tem como diretriz principal diagnosticar o código-fonte atual sob **quatro óticas estruturais**: a baixa coesão, o alto acoplamento, as responsabilidades mal distribuídas e as dificuldades de manutenção e testes. O **objetivo primário é mapear as falhas de projeto** que geram esses sintomas e, a partir dessa identificação, **propor refatorações** para saná-las. Tais melhorias serão baseadas em composição, no **Princípio de Demeter** e nos princípios SOLID, discutidos na obra *Clean Architecture* (Martin, 2019), visando aprimorar a modularidade, a flexibilidade e a qualidade do projeto, mantendo intacto o comportamento esperado do sistema.

O código-fonte base para a análise encontra-se no repositório público indicado no enunciado da atividade (https://github.com/vanessaborges2/refatoracao-solid). Para viabilizar o desenvolvimento das correções e melhorias, o projeto foi clonado para um repositório próprio (https://github.com/gabrielribeirof/solid-refactoring).

Este documento, inicialmente, apresenta os critérios e os fundamentos teóricos que embasaram a avaliação do código-fonte. Na sequência, o relatório expõe a análise individualizada e o diagnóstico detalhado para cada artefato do código, culminando nas respectivas propostas de solução.

2 CRITÉRIOS E FUNDAMENTOS DE ANÁLISE

O objetivo deste tópico é estabelecer os parâmetros que guiarão a avaliação. A seguir, são comentados os referenciais teóricos de cada pilar analítico requerido pelo enunciado.

2.1 Coesão

Segundo Pressman, a coesão no projeto orientado a objetos é definida como o "foco único" de um componente ou classe, implicando o “encapsulamento exclusivo de atributos e operações intimamente relacionados”. Nesta análise, buscaremos os níveis mais altos de “coesão funcional”, de “camadas” e de “comunicação”, categorias de coesão referenciadas pelo autor.

Para atingir e garantir essa alta coesão na prática, todo o conjunto de princípios de design SOLID será utilizado será utilizado como balizador, com destaque para o *Single Responsibility Principle*, que assegura que o componente possua um e apenas um motivo para ser modificado.

2.2 Acoplamento

O acoplamento é caracterizado por Pressman como uma “medida qualitativa do grau com que as classes estão ligadas entre si”. O autor adverte que o aumento do grau de conexão eleva proporcionalmente a complexidade do sistema.

A avaliação buscará identificar práticas nocivas, como o “acoplamento de conteúdo”, o “acoplamento de controle” e o “acoplamento externo”, categorias de acoplamento referenciadas pelo autor.

Para mitigar essas amarras e atingir baixo acoplamento, o *Dependency Inversion Principle* do SOLID será aplicado de forma sistemática para isolar os módulos de alto nível do modelo de domínio em relação às infraestruturas de baixo nível.

2.3 Responsabilidades mal distribuídas

A distribuição inadequada de responsabilidades será diagnosticada quando houver o acúmulo de funções incompatíveis em uma mesma unidade estrutural.

Violações ao *Open/Closed Principle* servirão como indicativos de que as classes assumiram mais papéis lógicos do que sua concepção original permitia, engessando a evolução do software e forçando modificações invasivas para a adição de novas regras.

2.4 Dificuldades de manutenção e testes

Para embasar a avaliação das dificuldades de manutenção e de testes, a teoria de Pressman estabelece uma relação de causa e efeito irrevogável: à medida que a

coesão diminui e o acoplamento aumenta, a dificuldade de implementação, testes e manutenção do software também aumenta.
O presente relatório irá assumir que atingir altos níveis de coesão e baixos níveis de acoplamento garante o alívio nas dificuldades de manutenibilidade e testabilidade de software.

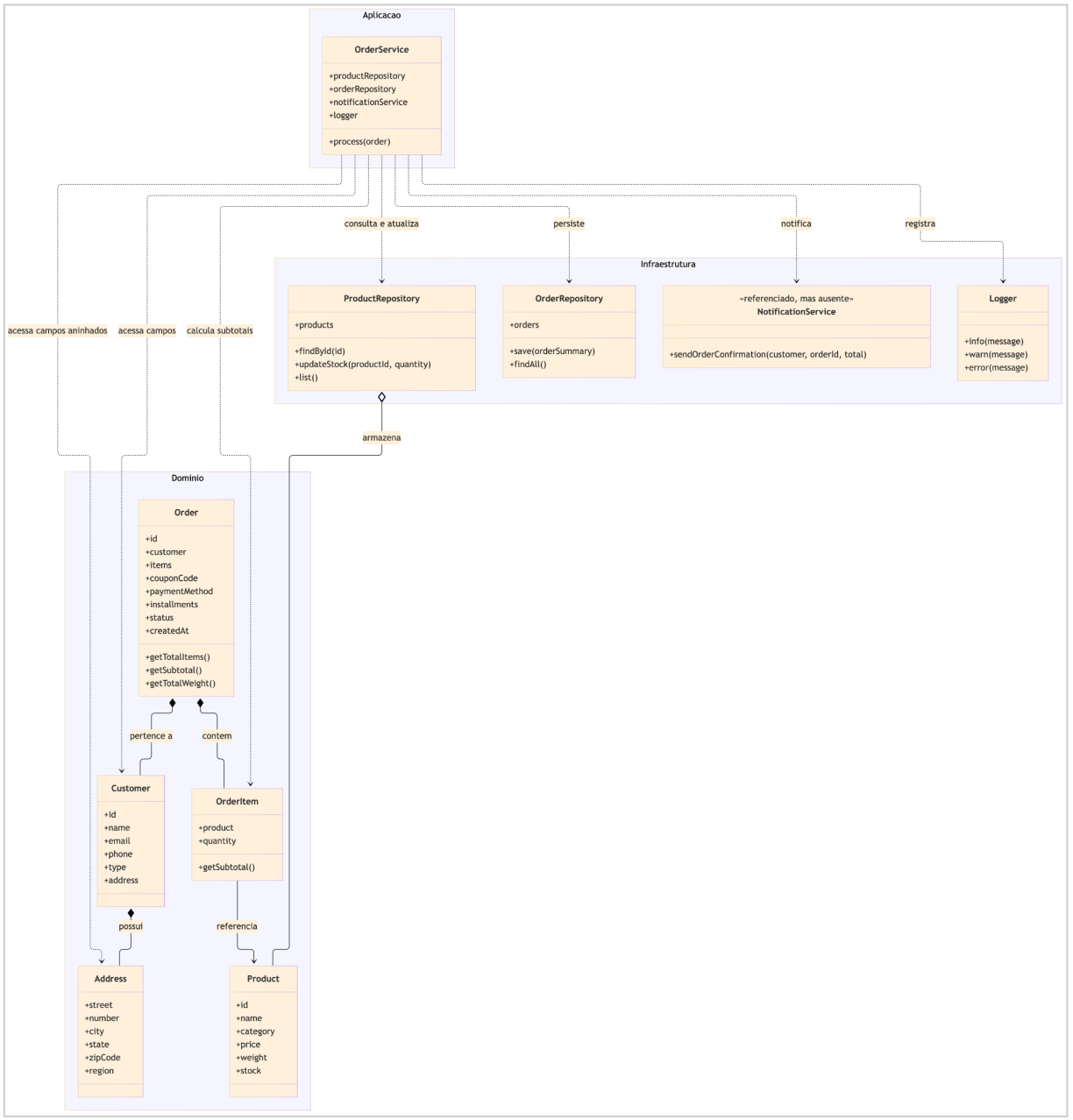
3 AMBIENTE E MÉTODO DE ANÁLISE

A preparação do projeto e a execução da análise seguiram as diretrizes estabelecidas na documentação original do código-fonte. O ambiente foi estruturado utilizando a plataforma Node.js, respeitando os requisitos mínimos de execução da aplicação.
A metodologia de inspeção consistiu em duas etapas objetivas. Inicialmente, realizou-se a tentativa de execução da rotina de inicialização para validar o comportamento e identificar falhas estruturais de montagem. Em seguida, foi feita com uma rigorosa revisão estática dos arquivos da aplicação. O código foi examinado sob a ótica dos critérios de coesão, acoplamento e distribuição de responsabilidades estabelecidos pela literatura de engenharia de software.
Essa abordagem viabilizou o mapeamento do fluxo de dependências e a identificação precisa das anomalias arquiteturais presentes no projeto legado.

4 VISÃO GERAL DO DESIGN ORIGINAL

O diagrama de classes a seguir reflete a organização estrutural e as relações de dependência do sistema antes da refatoração. Percebe-se três principais camadas: domínio, aplicação e infraestrutura.
Na camada de domínio, encontram-se as entidades do sistema. O diagrama ilustra que essas classes atuam majoritariamente como estruturas de dados, não tendo regras de negócio relevantes.
A camada de aplicação é inteiramente centralizada no componente *OrderService*. O modelo evidencia que este serviço orquestra o processo de forma monolítica.
Por fim, a camada de infraestrutura abriga os detalhes técnicos de persistência e operação, contemplando artefatos como repositórios e *logging*.

O diagrama também destaca a classe *NotificationService* como uma dependência ativamente referenciada pelo serviço principal, porém ausente no projeto original.



As conexões visuais confirmam o alto acoplamento arquitetural, comprovando que a camada de aplicação depende estritamente das implementações concretas e voláteis de infraestrutura.

5 DIAGNÓSTICO E SOLUÇÃO POR ARTEFATO

A seguir, a análise aplica os critérios estabelecidos para dissecar cada artefato de código, avaliando seu objetivo atual e diagnosticando as quatro métricas exigidas pelo enunciado.

5.1 Artefato da camada de serviço (*OrderService*)

5.1.1 Objetivo atual

Orquestrar o processamento do pedido, desde a validação de regras de negócio e cálculos financeiros até a persistência e exibição do resumo via saída padrão (stdout).

5.1.2 Baixa coesão

Carece de coesão funcional. O método *process* efetua dezenas de operações distintas (frete, estoque, taxas), perdendo seu foco único. Isso viola o SRP e o OCP, pois a classe absorve múltiplos motivos para mudar diante de novas regras.

5.1.3 Alto acoplamento

Apresenta acoplamento externo ao instanciar diretamente dependências de infraestrutura. Exibe também intenso acoplamento de controle, utilizando longas estruturas condicionais atreladas a tipos de pagamento ou regiões para ditar o fluxo de execução.

5.1.4 Solução

Fracionar o artefato utilizando composição, delegando regras específicas a serviços coesos (*InventoryService* para estoque, OrderSummaryService para resumo e *PaymentProcessor* composto pelas estratégias polimórficas PixPayment, *CreditCardPayment* e *BoletoPayment*, respeitando o LSP e o ISP através dos contratos enxutos supports e process). Para reduzir o acoplamento, aplica-se o DIP via Inversão de Controle [Fowler, 2004], injetando as dependências pelo construtor.

5.2 Artefatos da camada de domínio (*Order, Customer, Address, OrderItem, Product*)

5.2.1 Objetivo atual

Representa as estruturas de dados fundamentais para o tráfego de informações do pedido, do catálogo e do cliente.

5.2.2 Baixa coesão

Falham em atingir a coesão de comunicação. As operações que manipulam os dados estão implementadas fora das entidades, produzindo um modelo de domínio anêmico [Evans, 2016] e violando o SRP. Os objetos atuam como meros contêineres de variáveis.

5.2.3 Alto acoplamento

O sistema sofre de altíssimo acoplamento de conteúdo e violação direta ao Princípio de Demeter. O *OrderService* acessa propriedades profundamente aninhadas (ex.: *order.customer.address.region*), quebrando o encapsulamento que fundamenta o projeto orientado a objetos.

5.2.4 Solução

Enriquecer o domínio agrupando dados e operações na mesma classe, garantindo a coesão de comunicação e compondo objetos de valor dedicados, como *OrderPricing* (que encapsula os cálculos de subtotal, descontos, frete e taxas) e *OrderStatus*. O encapsulamento será restaurado aplicando o Princípio de Demeter por meio de métodos internos delegados (ex.: *getRegion()*, *decreaseStock()*), erradicando o acoplamento de conteúdo e ocultando a navegação estrutural.

5.3 Artefatos da camada de infraestrutura (*ProductRepository, OrderRepository, index.js*)

5.3.1 Objetivo atual

O *index.js* inicializa a aplicação, enquanto os repositórios mantêm o estado dos produtos em memória e salvam o histórico de transações.

5.3.2 Baixa coesão

O arquivo *index.js* carece de foco único. Ele mistura a configuração inicial de dependências com a execução imperativa de regras de negócio de um cliente fictício, falhando na coesão em camadas.

5.3.3 Alto acoplamento

Há forte acoplamento de dados na gestão de estado. O arquivo raiz popula o catálogo em uma instância de repositório, mas as baixas de estoque ocorrem em outra instância criada internamente no *OrderService*, gerando dessincronização.

5.3.4 Solução

Centralizar o *index.js* como o único ponto de montagem da aplicação, assegurando a coesão de camadas. A mesma instância de repositório criada no nível mais alto será injetada nas camadas inferiores via Inversão de Controle [Fowler, 2004], sanando o desvio de estado e estabilizando a arquitetura.

6 TESTES AUTOMATIZADOS E EVIDÊNCIAS DE EXECUÇÃO

Para assegurar a qualidade da refatoração e a preservação do comportamento esperado do sistema, foram implementadas duas suítes de testes automatizados utilizando o executor nativo do Node.js, totalizando 20 casos de teste. A primeira suíte contempla 19 testes de domínio, integração e design que validam os cálculos de precificação, descontos por cupom ou perfil, frete e modalidades de pagamento (PIX, cartão e boleto), bem como a proteção de invariantes e o encapsulamento de estado nas entidades. Essa bateria também verifica empiricamente a aplicação dos princípios SOLID por meio da injeção de dublês de teste (DIP) e da adição de uma nova estratégia de pagamento sem modificar a classe *PaymentProcessor* (OCP).

Complementarmente, a segunda suíte consiste em um teste de regressão que executa o ponto de entrada da aplicação (*src/index.js*) em um subprocesso isolado, normaliza o valor dinâmico da data de criação (*createdAt*) e confronta a saída padrão com o registo original do sistema legado, comprovando de forma determinística que o comportamento externo observável permaneceu inalterado.

```
> npm test

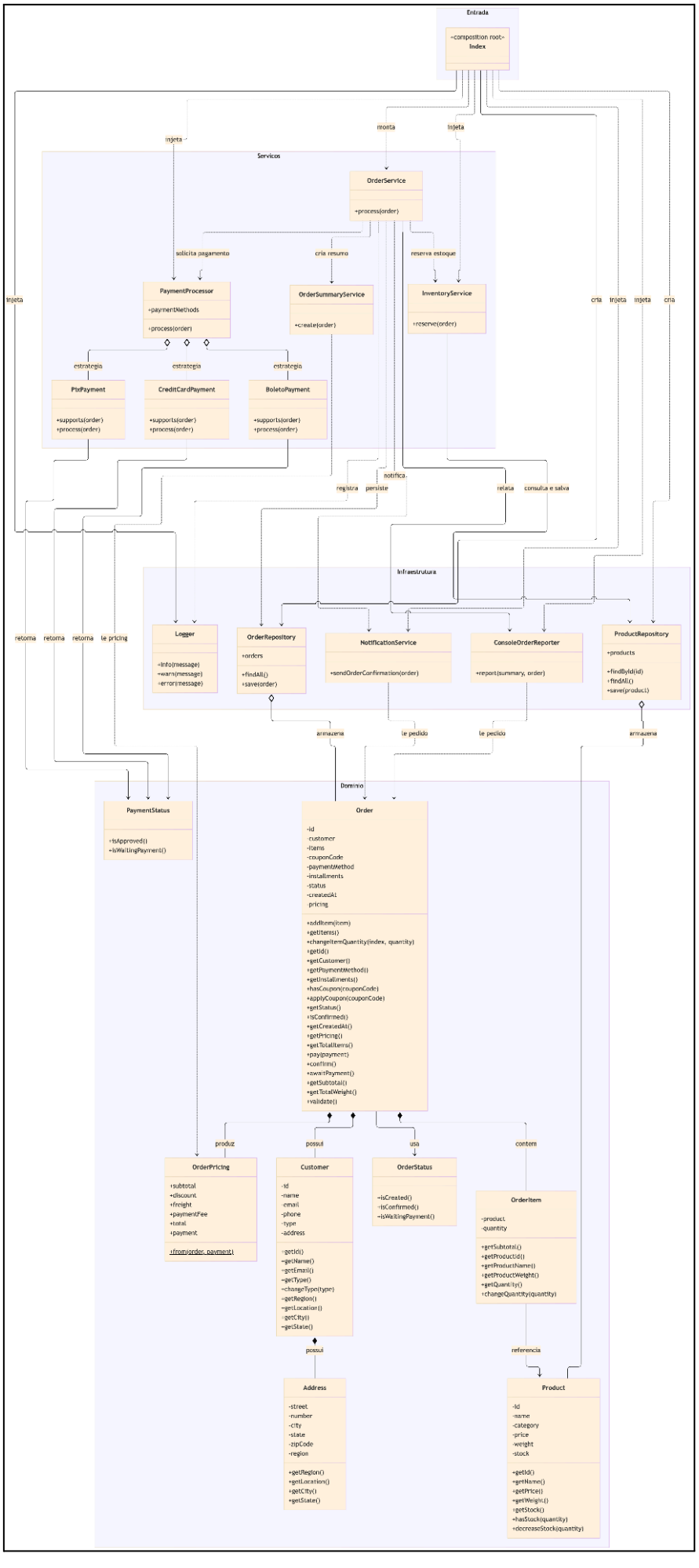
> arquitetura-solid-refatoracao-node@1.0.0 test
> node --test

✔ processa pedido aprovado e persiste o resumo (0.7905ms)
✔ Customer rejeita e-mail invalido na construcao (0.153166ms)
✔ mantem pedido aguardando pagamento para boleto (0.304625ms)
✔ rejeita estoque insuficiente antes de alterá-lo (0.110834ms)
✔ usa as dependências fornecidas pelo composition root (0.081583ms)
✔ publica o resumo pela porta de saída (0.085792ms)
✔ Order rejeita total invalido como regra de dominio (0.059875ms)
✔ Order calcula sua própria precificação de domínio (0.05375ms)
✔ PaymentProcessor aceita uma nova estrategia sem alterar o processador (0.094625ms)
✔ ProductRepository persiste alterações usando apenas save (0.075334ms)
✔ processa pagamento PIX com desconto e sem taxa (0.090542ms)
✔ processa cartão à vista sem taxa (0.077333ms)
✔ aplica frete grátis pelo cupom (0.059917ms)
✔ aplica desconto de funcionário (0.076125ms)
✔ aplica desconto fixo para pedidos acima de oito mil (0.074042ms)
✔ rejeita produto inexistente no estoque (0.065084ms)
✔ rejeita pedido sem cliente (0.033708ms)
✔ rejeita pedido sem itens (0.039625ms)
✔ impede alteração direta do estado encapsulado (0.029959ms)
✔ preserva exatamente a saida observavel do codigo original (52.602125ms)
ℹ tests 20
ℹ suites 0
ℹ pass 20
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 95.287917
```

7 CONCLUSÃO

A análise e a refatoração do sistema TechStore evidenciam a importância da aplicação de princípios consolidados de design de software. A reestruturação da arquitetura mitiga problemas de baixa coesão, alto acoplamento e responsabilidades mal distribuídas diagnosticados na versão original do código.

Com a introdução da Inversão de Controle, o enriquecimento comportamental da camada de domínio e a extração das lógicas para serviços especializados, o projeto atinge com êxito os objetivos da atividade. O sistema torna-se modular, testável e preparado para evoluções futuras, preservando de forma idêntica o seu comportamento esperado. A visão estrutural consolidada da nova arquitetura refatorada é ilustrada no diagrama de classes a seguir.


Entrada
«composition root»
Index
monta
injeta
injeta
Serviços
OrderService
+process(order)
solicita pagamento
cria resumo
reserva estoque
injeta
PaymentProcessor
+paymentMethods
+process(order)
OrderSummaryService
+create(order)
InventoryService
+reserve(order)
cria
injeta
injeta
cria
estrategia
estrategia
estrategia
PixPayment
+supports(order)
+process(order)
CreditCardPayment
+supports(order)
+process(order)
BoletoPayment
+supports(order)
+process(order)
notifica
registra
persiste
relata
consulta e salva
Infraestrutura
retorna
retorna
retorna
le pricing
Logger
+info(message)
+warn(message)
+error(message)
OrderRepository
+orders
+findAll()
+save(order)
NotificationService
+sendOrderConfirmation(order)
ConsoleOrderReporter
+report(summary, order)
ProductRepository
+products
+findById(id)
+findAll()
+save(product)
armazena
le pedido
le pedido
armazena
Domínio
PaymentStatus
+isApproved()
+isWaitingPayment()
Order
-id
-customer
-items
-couponCode
-paymentMethod
-installments
-status
-createdAt
-pricing
+addItem(item)
+getItems()
+changeItemQuantity(index, quantity)
+getId()
+getCustomer()
+getPaymentMethod()
+getInstallments()
+hasCoupon(couponCode)
+applyCoupon(couponCode)
+getStatus()
+isConfirmed()
+getCreatedAt()
+getPricing()
+getTotalItems()
+pay(payment)
+confirm()
+awaitPayment()
+getSubtotal()
+getTotalWeight()
+validate()
produz
possui
usa
contem
OrderPricing
+subtotal
+discount
+freight
+paymentFee
+total
+payment
+from(order, payment)
Customer
-id
-name
-email
-phone
-type
-address
+getId()
+getName()
+getEmail()
+getType()
+changeType(type)
+getRegion()
+getLocation()
+getCity()
+getState()
OrderStatus
+isCreated()
+isConfirmed()
+isWaitingPayment()
OrderItem
-product
-quantity
+getSubtotal()
+getProductId()
+getProductName()
+getProductWeight()
+getQuantity()
+changeQuantity(quantity)
possui
referencia
Address
-street
-number
-city
-state
-zipCode
-region
+getRegion()
+getLocation()
+getCity()
+getState()
Product
-id
-name
-category
-price
-weight
-stock
+getId()
+getName()
+getPrice()
+getWeight()
+getStock()
+hasStock(quantity)
+decreaseStock(quantity)

8 REFERÊNCIAS

EVANS, Eric. **Domain-Driven Design: atacando as complexidades no coração do software**. 3. ed. Rio de Janeiro: Alta Books, 2016.

FOWLER, Martin. **Inversion of Control Containers and the Dependency Injection pattern**. MartinFowler.com, 2004. Disponível em: https://martinfowler.com/articles/injection.html. Acesso em: 20 out. 2026.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Padrões de projetos: soluções reutilizáveis de software orientado a objetos**. Porto Alegre: Bookman, 2000.

LIEBERHERR, Karl J.; HOLLAND, Ian M. **Assuring good style for object-oriented programs**. IEEE Software, v. 6, n. 5, p. 38-48, set. 1989.

MARTIN, Robert C. **Arquitetura limpa: o guia do artesão para estrutura e design de software**. Rio de Janeiro: Alta Books, 2019.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.